Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Bu derste önce fonksiyon yazmanın ölçüsü ve matrislerin fonksiyonlara gönderilmesi konuşulur. Ardından seçim sistemi örnekleri, iki "doğrusal sıralama" algoritması (sayarak sıralama ve doğrudan yerleştirme) ve uygulama saati için asistanların hazırladığı sorular çözülür: dizinin ters çevrilmesi, çoğunluk elemanının bulunması (Boyer-Moore oylama algoritması), matrisin transpozu ve döndürülmesi, ikili bir matrisi simetrik yapmak için gereken değişiklik sayısı.
- Ne zaman fonksiyon yazılmalı?
  - Bir dizinin sıralanması veya yazdırılması gibi programda birden fazla yerde tekrar eden alt programlar (subroutine) fonksiyon olarak yazılmalıdır. Beş-altı satırlık bir kod iki kez tekrar ediyorsa bile fonksiyon olmalıdır. Böylece bir değişiklik tek yerde yapılır.
  - `printf` gibi zaten fonksiyon olan tek satırlık işler için ayrı bir fonksiyon yazmaya gerek yoktur.
  - Fonksiyon çağrısı programın akışını keser ve bazı yedekleme işlemleri gerektirir. Sistem programlama veya alt seviye programlama yapılıyorsa ve kod tekrar etmiyorsa fonksiyon yazmak gerekmez. Günümüz derleyicileri ve işlemcileri bu maliyeti büyük ölçüde azaltır. Nesneye yönelik programlamada her iş için ayrı fonksiyon yazılır.
  - `#define` ile yazılan makrolar da kodu kısaltır. Ayrıntıları gelecek dönemdedir.
- Diziye ve matrise ilk değer verme
  - `int dizi[10] = {1, 2, 4};` ilk üç elemanı verir, kalanlar 0 olur. Bütün diziyi sıfırlamak için `= {0}` yeterlidir. Bunun için `static` kullanmaya gerek yoktur.
  - Matris de aynı şekilde `int M[5][5] = {0};` ile sıfırlanabilir.
- Matrislerin fonksiyonlara gönderilmesi
  - Bu ders için önerilen yol: `void yazdir(int M[][5], int m, int n)`. Sütun sayısı yazılır, satır sayısı yazılmaz. Satır ve sütun sayıları ayrı parametrelerle gelir.
  - Pointer ile de yapılabilir. Derste `*(*(M + i) + j)` biçimi denendi: `M + i` i. satırın adresini, `*(M + i) + j` o satırdaki j. hücrenin adresini verir. Bu yazımlarla ve tiplerle ilgili derleyici uyarıları ve hatalar çıktı. Hoca bu ders kapsamında birinci yazımın yeterli olduğunu söyler.
- C ve C++ uyarısı
  - Kod C dosyası olarak (`.c`) kaydedilmelidir. C++ C'yi kapsar gibi görünse de derleyiciler açısından farklıdır.
- Seçim sistemi örnekleri (V vatandaş, P parti)
  - 1. durum: her vatandaş tek bir partiye oy verir. Vatandaş dizisinde `vatandas[i]` i. vatandaşın oy verdiği partidir. Parti dizisi sıfırlanır ve tek döngüyle `parti[vatandas[i]]++` yapılır. Kazanan, parti dizisindeki en büyük değerdir.
  - Sonuçların yatay histogramı: dış döngü partileri, iç döngü 1'den `parti[i]`'ye kadar gezer ve yıldız basar. Her partiden sonra alt satıra geçilir. Histogram bir veri görselleştirme aracıdır: çok sayıda değer arasından en yükseği bir bakışta görülür.
  - 2. durum: her vatandaş K partiye oy verir. Bir dizi yetmez, matris gerekir: `O[i][j]`, i. vatandaşın j. oyunu verdiği partidir (V × K). Transpozu (K × V) da aynı işi görür, bu bir tasarım tercihidir. İki iç içe döngüyle `parti[O[i][j]]++`.
  - 3. durum: her vatandaş her partiye puan verir (ör. 1-100). Matris V × P olur: `O[i][j]`, i. vatandaşın j. partiye verdiği puandır. Toplam puan için `puan[j] += O[i][j]`. Puanların geçerli aralıkta girilmesi `if` veya `do-while` ile kontrol edilebilir.

  - Bu durumda bazı öğrenciler sonuçları önce bir matriste toplayıp sonra diziye çevirir. Problem bir diziyle çözülebiliyorsa matris kullanmak iyi bir algoritma değildir.
  - Kazananı bulmak için puan dizisinin en büyüğü bulunur (n adım). Aynı en yüksek puanı birden fazla parti aldıysa dizi bir kez daha gezilir ve hepsi yazdırılır (koalisyon).
  - 4. durum: her vatandaşın verdiği puan bir ölçüte göre ağırlıklandırılır (ör. bir f fonksiyonuyla IQ'ya veya eğitim durumuna göre). Bu durumda matris `float` olur.
  - Bir öğrenci 2. ve 3. durumu matris kullanmadan, girilen puanları doğrudan parti dizisine ekleyerek çözmeyi önerdi. Hoca, yalnızca toplamlar isteniyorsa bunun olacağını, ama vatandaşların hangi partiye ne verdiği bilgisi saklanmak isteniyorsa matris gerektiğini söyler.
- Sayarak doğrusal sıralama (linear sort with counting)
  - Fikir: bir elemandan küçük k tane eleman varsa, o elemanın sıralı dizideki yeri k + 1'dir. C'de indisler 0'dan başladığı için yer doğrudan k'dır.
  - Örnek: 3, 1, 4, 8. 3'ten küçük 1, 1'den küçük 0, 4'ten küçük 2, 8'den küçük 3 eleman vardır. Sonuç: 1, 3, 4, 8.
  - 1. sürüm: her eleman için diğer bütün elemanlara bakılır: `if (dizi[i] > dizi[j]) kucuk[i]++;`, sonra `yeni[kucuk[i]] = dizi[i];`. 8 4 5 3 99 doğru sıralandı. Ama eşit elemanlar aynı yere yazılır ve bir hücre boş kalır. Ayrıca n^2 karşılaştırma ve iki ek dizi gerekir. Bu yüzden bubble sort veya selection sort'tan daha iyi değildir.
  - 2. sürüm: `j i + 1`'den başlar. `dizi[i] > dizi[j]` ise `kucuk[i]++`, değilse `kucuk[j]++`. Her çift bir kez karşılaştırılır.
  - Karşılaştırma sayısı (temel işlem `if`): 1. sürümde $\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} 1 = n^2$. 2. sürümde $\sum_{i=0}^{n-1}(n-1-i) = n(n-1) - \frac{n(n-1)}{2} = \frac{n(n-1)}{2} \approx \frac{n^2}{2}$. Yani matrisin yalnızca bir üçgeni taranır. Ek bellek ihtiyacı aynıdır.
- Doğrudan yerleştirerek doğrusal sıralama
  - Önce dizinin en büyük elemanı bulunur (n adım). 0'dan en büyük elemana kadar indisleri olan bir B dizisi açılır ve bütün elemanları -1 yapılır (elemanların 0 veya pozitif olduğu kabul edilir).
  - Her eleman kendi değerindeki indise yazılır: `B[A[i]] = A[i]` (n adım). Sonra B baştan sona gezilir ve -1 olmayanlar sırayla A'ya kopyalanır.
  - Toplam yaklaşık $3n$ işlem. Çok hızlıdır, ama en büyük eleman büyükse (ör. $10^6 - 5$) korkunç büyüklükte bir bellek ayırmak gerekir.
  - Ödev: algoritmayı kodlayın ve bu sorunlara (büyük değerler, aynı değerden birden fazla eleman) çözümler önerin. İsterseniz sayarak sıralamayla birleştirin.
- Uygulama soruları: dizinin ters çevrilmesi
  - En basit yol: yeni bir diziye sondan başa kopyalamak. Ek bellek ve n işlem gerekir.
  - Derste görülen yol: `for (i = 0; i < n/2; i++)` ile `dizi[i]` ve `dizi[n-i-1]` swap edilir. C'de son eleman `n - 1`'dir, `- 1` unutulursa dizinin dışına yazılır.
  - Asistanların çözümü: `left = 0`, `right = n - 1`, `while (left < right) { swap; left++; right--; }`. İkili aramaya benzer bir yapı.
  - Hocanın kaba işlem sayımı: `for` sürümünde her turda indis hesapları (`n - i - 1`) ve `n/2` karşılaştırması yüzünden yaklaşık $11 \cdot \frac{n}{2}$ artı döngü kontrolü, `while` sürümünde yaklaşık $9 \cdot \frac{n}{2} + 4$. İkisi de n mertebesindedir. `while` sürümü bir ek değişken (`right`) kullanır ama daha az işlem yapar. Gerçek farkı görmek için büyük dizilerde aynı bilgisayarda süre ölçülebilir.
- Uygulama soruları: çoğunluk elemanı (majority element)
  - Çoğunluk elemanı, dizide $n/2$'den fazla geçen elemandır. Örnek dizide 3, 8 elemanın 5'inde geçiyor.
  - 1. çözüm (asistanların ilk çözümü): her eleman için iç döngüyle frekansını say ve en büyük frekansı tut. Sonra en büyük frekans $n/2$'den büyük mü bak. n^2 işlem, ek dizi yok.
  - 2. çözüm (Boyer-Moore çoğunluk oylaması, derste "Moore voting" olarak geçti): ilk eleman aday, sayaç 1. Her elemanda aday ile aynıysa sayaç artar, değilse azalır. Sayaç 0 olursa o anki eleman yeni aday olur ve sayaç 1 yapılır. Sonra adayın gerçek frekansı ikinci bir döngüyle sayılır ve $n/2$ ile karşılaştırılır. $2n$ işlem.
  - Derste 2 3 3 4 3 3 girildi ve aday 3, frekansı 4 bulundu.
  - Bir öğrenci, baştaki aynı elemanlardan sonra hep farklı değerler gelirse adayın yanlış çıkacağını sordu. Hoca adımları izledi: `3 3 3 4 5 6 7 8 9 10` dizisinde son aday 10,

frekansı 1 olur. Hoca ve öğrenciler, algoritmanın çoğunluk elemanı varsa onu garanti ettiğini, ama en sık geçen elemanı (mod) her zaman bulmadığını gördü. Çoğunluk varsa da sayaç yol üzerinde sıfırlanabilir (eleman dağılmışsa); kesin olan, son adayın o eleman çıkmasıdır.
  - Hoca $n/2$ şartı olmadan en sık elemanı tek değişkenli bir yedekle bulmayı denedi, ama önceki frekansların kaybolduğunu ve bunun için bir dizi gerektiğini gösterdi.
  - `do { scanf("%d", &n); } while (n > MAX);` ile dizinin boyutu `MAX`'ı aşmayacak şekilde alınır. `do-while`'ın sonundaki noktalı virgül unutulmamalıdır.
- Uygulama soruları: matrisler
  - Transpoz: `T[j][i] = A[i][j]`. M × N bir matrisin transpozu N × M'dir. Kare olmayan matrislerde döngü sınırlarına dikkat edilmelidir.
  - Çalışma sorusu: bir matrisi (görüntüyü) 90 derece döndüren kodu yazın. Hoca ayrıca satırları ters çevirmeyi ve matrisi aynalamayı da örnek verir.
  - İkili (0-1) bir kare matrisi köşegene göre simetrik yapmak için kaç bit değiştirmek gerekir?
    * 1. çözüm: matrisin transpozunu al, iki matrisi eleman eleman karşılaştır, farkları say ve 2'ye böl (her uyuşmazlık iki kez sayılır). n^2 işlem ve n^2 ek bellek.
    * 2. çözüm: transpoz almadan yalnızca alt üçgeni gez: `for (i = 0; i < n; i++) for (j = 0; j < i; j++) if (A[i][j] != A[j][i]) flip++;`. 2'ye bölmeye gerek yoktur. Yaklaşık $\frac{n^2}{2}$ işlem ve ek bellek yok.
    * Burada `while` yerine `for` kullanılır, çünkü yalnızca simetrik mi diye bakılmıyor, değiştirilmesi gereken bütün hücreler sayılıyor. Yalnızca simetrik olup olmadığını kontrol etmek için ilk uyuşmazlıkta duran `while` daha iyidir.
    * Bir öğrenci üst ve alt üçgen toplamlarının farkına bakmayı önerdi. Hoca bir karşıt örnekle (bir yanda 1/0, diğer yanda 0/1) toplamların eşit olup matrisin yine de simetrik olmayabileceğini gösterdi.
    * `j`, `i + 1`'den `n`'e giderse üst üçgen gezilir. Bu da aynı sonucu verir.
- Bir sonraki ders
  - Döngülerle yazılmış (iteratif) algoritmaların analizi ve rastgele sayı üretimi işlenecektir.

Hocanın Özellikle Vurguladığı Kısımlar

- Tekrar eden kod parçaları fonksiyona alınmalıdır. Ama tek satırlık işler için fonksiyon yazmak gereksizdir.
- Bir matrisi fonksiyona göndermek için bu ders kapsamında `int M[][SUTUN]` yazımı yeterlidir.
- Laboratuvarda C dosyası gönderin, C++ değil.
- Önce hangi veri yapısının kullanılacağını belirleyin. Problem bir diziyle çözülebiliyorsa matris kullanmak gereksizdir.
- Algoritmaları analiz ederken temel işlem (çoğunlukla karşılaştırma) seçilir ve toplam sembolleriyle kaç kez yapıldığı hesaplanır.
- Simetri kontrolünde önce simetrik matrisin tanımına dönün: matris transpozuna eşittir. Basit çözümden başlayıp iyileştirin.

Kısa Tekrar Notları

- Matris parametresi: `int M[][5], int m, int n`.
- Tek oylu seçim: `parti[vatandas[i]]++`. K oylu: `parti[O[i][j]]++`. Puanlı: `puan[j] += O[i][j]`.
- Sayarak sıralama: `kucuk[i]` = i. elemandan küçüklerin sayısı, `yeni[kucuk[i]] = dizi[i]`. j $i+1$'den başlarsa $\frac{n(n-1)}{2}$ karşılaştırma.
- Doğrudan yerleştirme: `B[A[i]] = A[i]`, $O(n)$ ama en büyük değer kadar bellek.
- Ters çevirme: `while (left < right) swap(left++, right--)`.
- Boyer-Moore: aday + sayaç, sonra adayı say. Yalnızca $n/2$'den fazla geçen eleman varsa garanti.
- Simetri için değişiklik: alt üçgende `A[i][j] != A[j][i]` say.

Detaylı Açıklamalar

Sayarak sıralamanın iki sürümü

Derste yazılan sürümler fonksiyon hâline getirilip ders sonrasında `gcc -Wall -std=c89 -pedantic` ile denendi:

```
/* 1. surum: her eleman icin butun diziye bakilir */
for (i = 0; i < MAX; i++)
    for (j = 0; j < MAX; j++)
        if (dizi[i] > dizi[j])
            kucuk[i]++;

/* 2. surum: her cift bir kez karsilastirilir */
for (i = 0; i < MAX; i++)
    for (j = i + 1; j < MAX; j++)
        if (dizi[i] > dizi[j])
            kucuk[i]++;
        else
            kucuk[j]++;

for (i = 0; i < MAX; i++)
    yeni[kucuk[i]] = dizi[i];
```

- 8 4 5 3 99 dizisi 1. sürümle 3 4 5 8 99 olarak sıralandı.
- 3 5 3 2 6 dizisinde 1. sürümde iki 3'ün `kucuk` değeri aynı (1) çıktı. İkisi de `yeni[1]`'e yazıldı ve `yeni[2]` boş kaldı. Derste görülen hata buydu.
- Aynı dizi 2. sürümle 2 3 3 5 6 olarak doğru sıralandı.

Boyer-Moore çoğunluk oylaması

```
maxi = 0;
count = 1;
for (i = 1; i < n; i++) {
    if (dizi[maxi] == dizi[i])
        count++;
    else
        count--;
    if (count == 0) {
        maxi = i;
        count = 1;
    }
}
candidate = dizi[maxi];
count = 0;
for (i = 0; i < n; i++)
    if (dizi[i] == candidate)
        count++;
if (count > n / 2)
    printf("%d\n", candidate);
else
    printf("no majority\n");
```

Ders sonrasında 2 3 3 4 3 3 dizisi için 3 adayı ve 4 frekansı bulundu. $4 > 6/2$ olduğu için 3 çoğunluk elemanıdır.

Ders sonrası notlar

- **Sayarak sıralamada eşit elemanlar:** Derste her iki sürümde de eşit elemanlarda doğruluk sorunu olduğu söylendi. Bu sorun yalnızca 1. sürümde vardır. 2. sürümde eşitlik durumu `else` koluna düşer ve eşit iki elemandan sonra gelenin sayacı artar. Böylece eşit

elemanlar farklı yerlere yazılır. Ders sonrasındaki denemede 2. sürüm 3 5 3 2 6'yı doğru sıraladı. Bu yönteme literatürde karşılaştırma sayarak sıralama (comparison counting sort) denir. Değer aralığına dayanan "counting sort" ise farklı bir algoritmadır (Ders 10'daki frekans hesabına benzer).

- **Çoğunluk şartı:** Derste kodda `count >= n/2` kullanıldı. Çoğunluk tanımı "$n/2$'den fazla"dır, bu yüzden şart `count > n/2` olmalıdır. Örneğin 6 elemanlı bir dizide tam 3 kez geçen eleman çoğunluk değildir. Boyer-Moore algoritması da yalnızca $n/2$'den fazla geçen bir eleman varsa onu kesin olarak bulur.
- **"Longest common subsequence":** Hoca, ardışık olarak en uzun süre tekrar eden elemanı bulma problemine "longest common subsequence" dedi. Bu ad iki dizinin en uzun ortak alt dizisi problemine aittir. Tek bir dizide arka arkaya tekrar eden en uzun bölümü bulmak, "en uzun tekrar (run)" problemidir ve tek döngüyle çözülür.
- **Matrisi döndürme ve aynalama:** Derste 1..9 matrisinin 90 derece döndürülmüş hâli "3, 2, 1, 6, 5, 4, 9, 8, 7" diye okundu. Hoca transpozun ilk sütunuyla karşılaştırdığı için bu büyük olasılıkla sütun sütun okumadır ve saat yönünün tersine 90 derece döndürme doğrudur. Satır satır okunursa her satırın ters çevrilmesidir (yatay aynalama). 90 derece saat yönünde döndürme 7 4 1 / 8 5 2 / 9 6 3, saat yönünün tersine döndürme 3 6 9 / 2 5 8 / 1 4 7 verir. Hocanın "180 derece aynalama" dediği 7 8 9 / 4 5 6 / 1 2 3 dikey aynalamadır. 180 derece döndürme 9 8 7 / 6 5 4 / 3 2 1'dir. Saat yönünde 90 derece döndürme, transpoz alınıp her satırın ters çevrilmesiyle elde edilebilir.
- **Matrisi `int *` ile göndermek:** Matris tek bir `int *` olarak gönderilirse (`&M[0][0]`), elemana `M[i * n + j]` ile erişilir. Derste denenen `M[i + j]` yazımı yanlış hücreyi verir, çünkü satır atlamak için satır uzunluğuyla çarpmak gerekir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.